

PERTEMUAN 2 • DATA SCIENCE & MACHINE LEARNING

# Perkakas Data

pandas & Visualisasi Data dengan Python

*“Tidak ada model yang baik tanpa data yang rapi. Sebelum belajar melatih model, kita belajar menjinakkan datanya.”*

# Tujuan Pembelajaran

*Setelah pertemuan ini, mahasiswa diharapkan mampu:*

1

## Menjelaskan

Membedakan Series, DataFrame, dan Index serta kapan masing-masing dipakai.

2

## Mengoperasikan

Melakukan seleksi, penyaringan, dan modifikasi data dengan loc, iloc, dan [ ].

3

## Meringkas

Menggunakan groupby, agregasi, pivot\_table, dan penggabungan antar-tabel.

4

## Memvisualkan

Membuat grafik eksploratif dengan Plotly dan membacanya secara kritis.

# Mengapa Kita Mulai dari Data?

*80% waktu proyek machine learning habis untuk menyiapkan data, bukan melatih model.*

01

## Data adalah fondasi

Setiap model bermula dan berakhir pada data. Model hanya sebaik data yang melatihnya — garbage in, garbage out.

02

## Input & eksperimen

Keberhasilan eksperimen bergantung pada kemampuan kita menyiapkan input dan membaca output secara sistematis.

03

## Keterampilan wajib

Baik di laboratorium riset maupun industri, kemampuan data wrangling dan visualisasi adalah syarat dasar.

*Analogi: data mentah itu seperti bahan pasar. pandas adalah pisau dan talenannya — tanpa persiapan, masakan (model) tidak akan jadi.*

# Peta Materi Hari Ini

*Delapan bagian, dari struktur data sampai visualisasi.*

1

**Struktur Data pandas**

2

**Eksplorasi DataFrame**

3

**Seleksi & Pengambilan  
Data**

4

**Penyaringan Data  
(Filtering)**

5

**Modifikasi DataFrame**

6

**Agregasi &  
Pengelompokan**

7

**Menggabungkan  
DataFrame**

8

**Visualisasi Data**

*Bagian 1–7 memakai pandas; bagian 8 memakai Plotly. Semua contoh memakai satu dataset yang sama agar mudah diikuti.*

## BAGIAN 01

# Struktur Data pandas

*Mengenal Series, DataFrame, dan Index sebagai tiga blok penyusun pandas.*

- 1. Struktur Data pandas
- 2. Eksplorasi DataFrame
- 3. Seleksi & Pengambilan Data
- 4. Penyaringan Data (Filtering)
- 5. Modifikasi DataFrame
- 6. Agregasi & Pengelompokan
- 7. Menggabungkan DataFrame
- 8. Visualisasi Data

# Apa Itu pandas?

*Nama pandas berasal dari “panel data” — istilah ekonometrika untuk data tabel antarwaktu.*

## Dengan pandas kita dapat:

- Menyusun data dalam bentuk tabel (baris × kolom).
- Menyaring informasi berdasarkan kondisi tertentu.
- Mengolah data untuk menghasilkan informasi baru.
- Memakai operasi numerik NumPy pada data.
- Melakukan komputasi tervektorisasi yang jauh lebih cepat daripada perulangan biasa.

```
import pandas as pd
import numpy as np

pd.__version__
```

## Mengapa bukan Excel?

Kode pandas dapat diulang, diperiksa, dan dijalankan pada jutaan baris. Klik-klik di Excel tidak meninggalkan jejak yang bisa direproduksi orang lain.

*pandas adalah perkakas standar di dunia riset maupun industri untuk data tabular.*

# Tiga Objek Inti pandas

*Pahami ketiganya lebih dulu — sisa materi hanyalah operasi di atas ketiga objek ini.*

## Series

Larik satu dimensi berlabel. Satu kolom tabel adalah sebuah Series.

```
s = pd.Series([30, 307, 80])
```

## DataFrame

Tabel dua dimensi. Kumpulan Series yang berbagi Index yang sama.

```
df = pd.DataFrame(data)
```

## Index

Label baris (atau kolom). Bukan sekadar nomor — bisa berupa teks atau tanggal.

```
df.index
```

**Ingat:** DataFrame = beberapa Series + satu Index bersama. Karena itu banyak metode Series juga tersedia pada DataFrame.

# Series: Larik Satu Dimensi Berlabel

*Sebuah Series terdiri atas dua komponen: values dan index.*

```
import pandas as pd

s = pd.Series(["selamat", "datang", "di", "Data
Science"])
```

## **s.values**

```
array(['selamat', 'datang', 'di', 'Data Science'], dtype=object)
```

## **s.index**

```
RangeIndex(start=0, stop=4, step=1)
```

## **s[1]**

```
'datang'
```

## Poin penting

- values menyimpan datanya, index menyimpan labelnya.
- Index default berupa angka 0, 1, 2, ... tetapi dapat diganti menjadi teks atau tanggal.
- Semua elemen Series idealnya bertipe sama (dtype) agar operasi tervektorisasi berjalan cepat.
- Operasi aritmetika berjalan elemen per elemen tanpa perlu perulangan for.



# DataFrame: Tabel yang Kita Pakai Sehari-hari

Contoh dataset yang akan dipakai sepanjang kuliah ini: landmark kampus.

```
data = {
    'Nama': ['Gerbang', 'Menara', 'Perpustakaan',
            'Lapangan', 'Plaza'],
    'Jenis': ['Gerbang', 'Menara', 'Perpustakaan',
            'Ruang Terbuka', 'Plaza'],
    'Tinggi': [12, 74, 28, 0, 0],
    'Tahun': [1960, 1975, 1968, None, 1992]
}
df = pd.DataFrame(data)
```

|   | Nama         | Jenis         | Tinggi | Tahun |
|---|--------------|---------------|--------|-------|
| 0 | Gerbang      | Gerbang       | 12     | 1960  |
| 1 | Menara       | Menara        | 74     | 1975  |
| 2 | Perpustakaan | Perpustakaan  | 28     | 1968  |
| 3 | Lapangan     | Ruang Terbuka | 0      | NaN   |
| 4 | Plaza        | Plaza         | 0      | 1992  |

## Empat cara membuat DataFrame

① dari berkas CSV · ② dari dictionary · ③ dari list + nama kolom · ④ dari beberapa Series

Perhatikan nilai None pada kolom Tahun: pandas menampilkannya sebagai NaN. Kita akan menanganinya pada Bagian 5.

# Membaca Data dari Berkas

*Dalam praktik, data hampir selalu datang dari berkas atau basis data.*

## CSV

```
pd.read_csv('data/landmark.csv')
```

Format paling umum. Perhatikan pemisah (sep) dan encoding.

## Excel

```
pd.read_excel('data.xlsx', sheet_name=0)
```

Butuh pustaka openpyxl. Pilih sheet yang tepat.

## JSON / SQL

```
pd.read_json(...) · pd.read_sql(...)
```

Untuk data dari API atau basis data relasional.

**Tip:** `index_col='Tahun'` menjadikan kolom Tahun sebagai Index, berguna untuk data deret waktu.

## BAGIAN 02

# Eksplorasi DataFrame

*Sebelum menganalisis, kenali dulu bentuk dan isi datanya.*

- 1. Struktur Data pandas
- **2. Eksplorasi DataFrame**
- 3. Seleksi & Pengambilan Data
- 4. Penyaringan Data (Filtering)
- 5. Modifikasi DataFrame
- 6. Agregasi & Pengelompokan
- 7. Menggabungkan DataFrame
- 8. Visualisasi Data

# Fungsi Utilitas untuk Mengenal Data

*Enam perintah pertama yang selalu dijalankan seorang data scientist.*

**head() / tail()**

Melihat beberapa baris pertama atau terakhir.

**info()**

Tipe data tiap kolom dan jumlah nilai non-null.

**describe()**

Ringkasan statistik kolom numerik.

**sample()**

Mengambil baris secara acak agar tidak bias urutan.

**value\_counts()**

Menghitung frekuensi tiap nilai unik.

**unique()**

Menampilkan daftar nilai unik.

*Kebiasaan baik: jalankan `df.head()`, `df.info()`, lalu `df.describe()` setiap kali membuka dataset baru.*

# head(), tail(), shape, dan size

*Empat perintah untuk menjawab: “sebesar apa dan seperti apa data saya?”*

**df.head()**

5 baris pertama (nilai bawaan n = 5)

**df.tail(3)**

3 baris terakhir

**df.shape**

(5, 4) → 5 baris, 4 kolom

**df.size**

20 → jumlah seluruh sel (5 × 4)

## Kesalahan yang sering terjadi

- Menulis df.shape() dengan tanda kurung. shape adalah atribut, bukan metode — tulis df.shape saja.
- Menganggap size sama dengan jumlah baris. size adalah baris × kolom; untuk jumlah baris gunakan len(df).
- Melihat head() lalu menyimpulkan seluruh data. Lima baris pertama sering terurut dan tidak mewakili keseluruhan — gunakan sample().

# sample(): Mengambil Baris Secara Acak

*Lebih jujur daripada head() karena tidak terpengaruh urutan data.*

```
.sample(n=3)
```

Mengambil 3 baris secara acak.

```
.sample(frac=0.2)
```

Mengambil 20% dari seluruh baris.

```
.sample(n=3, replace=True)
```

Boleh mengambil baris yang sama lebih dari sekali.

```
.sample(n=3, random_state=42)
```

Hasil acak yang sama setiap dijalankan (reproducible).

*Selalu gunakan random\_state pada tugas dan laporan. Tanpa itu, hasil Anda tidak dapat direproduksi oleh dosen maupun rekan setim.*

# value\_counts() dan unique()

*Dua perintah kunci untuk memahami kolom kategorikal.*

## **Series.value\_counts()**

Menghitung berapa kali setiap nilai unik muncul, terurut dari yang terbanyak.

```
df['Jenis'].value_counts()
```

```
Gerbang      1
Menara        1
Perpustakaan  1
Ruang Terbuka 1
Plaza         1
Name: count, dtype: int64
```

## **Series.unique()**

Mengembalikan larik berisi setiap nilai unik, tanpa menghitung frekuensinya.

```
df['Jenis'].unique()
```

```
array(['Gerbang', 'Menara',
       'Perpustakaan',
       'Ruang Terbuka', 'Plaza'])
```

*Butuh jumlahnya saja? Pakai df['Jenis'].nunique().*

## BAGIAN 03

# Seleksi & Pengambilan Data

*Tiga cara mengambil potongan data: berdasarkan posisi, label, dan konteks.*

- 1. Struktur Data pandas
- 2. Eksplorasi DataFrame
- **3. Seleksi & Pengambilan Data**
- 4. Penyaringan Data (Filtering)
- 5. Modifikasi DataFrame
- 6. Agregasi & Pengelompokan
- 7. Menggabungkan DataFrame
- 8. Visualisasi Data



# `iloc[]` — Seleksi Berdasarkan Posisi

*Bacalah sebagai “integer location”: yang dipakai adalah nomor urut, bukan label.*

- Argumen dapat berupa daftar, irisan (slice), atau satu nilai tunggal.
- Konvensi Python: posisi pertama adalah 0.
- Irisan bersifat eksklusif — batas kanan tidak ikut terambil.

```
df.iloc[[1, 2]]
```

Baris ke-1 dan ke-2 (semua kolom).

```
df.iloc[2:4]
```

Baris 2 dan 3 — baris ke-4 TIDAK ikut.

```
df.iloc[2]
```

Satu baris utuh → hasilnya sebuah Series.

```
df.iloc[[1, 2], [1, 2, 3]]
```

Baris 1–2, kolom 1 sampai 3.

```
df.iloc[:, 1:2]
```

Semua baris, hanya kolom ke-1.

```
df.iloc[0, 1]
```

Satu sel tunggal → hasilnya nilai tunggal.

# loc[] — Seleksi Berdasarkan Label

*Yang dipakai adalah label baris dan nama kolom, bukan nomor urut.*

- Argumen dapat berupa daftar, irisan, atau satu nilai tunggal.
- Berbeda dengan iloc: irisan pada loc bersifat INKLUSIF.
- Label baris tidak harus angka — bisa nama, kode, atau tanggal.

```
df.loc[[1, 2]]
```

Baris berlabel 1 dan 2.

```
df.loc[2:3]
```

Baris 2 DAN 3 — batas kanan ikut.

```
df.loc[2]
```

Satu baris utuh sebagai Series.

```
df.loc[[1, 2], ['Jenis', 'Tinggi']]
```

Baris 1–2, dua kolom terpilih.

```
df.loc[:, 'Nama':'Tinggi']
```

Semua baris, kolom Nama sampai Tinggi.

```
df.loc[0, 'Jenis']
```

Satu sel: baris 0, kolom Jenis.

# Kurung Siku [ ] — Seleksi Bergantung Konteks

*[ ] hanya menerima satu argumen, dan artinya berubah mengikuti jenis argumen itu.*

**Irisan angka**

```
df[1:3]
```

Diartikan sebagai BARIS ke-1 dan ke-2.

**Daftar label**

```
df[['Nama', 'Tahun']]
```

Diartikan sebagai KOLOM. Hasilnya DataFrame.

**Label tunggal**

```
df['Tahun']
```

Satu KOLOM. Hasilnya Series.

*Karena ambigu, [ ] nyaman untuk penulisan cepat, tetapi gunakan loc / iloc bila ingin kode yang jelas dibaca.*

# Ringkasan: loc vs iloc vs [ ]

*Satu tabel yang layak Anda hafalkan.*

| Aspek              | loc[]               | iloc[]                  | [ ]                   |
|--------------------|---------------------|-------------------------|-----------------------|
| Dasar seleksi      | Label               | Posisi (angka)          | Bergantung konteks    |
| Batas kanan irisan | Ikut terambil       | Tidak ikut              | Tidak ikut (baris)    |
| Menerima Boolean   | Ya                  | Terbatas                | Ya                    |
| Jumlah argumen     | Dua (baris, kolom)  | Dua (baris, kolom)      | Satu saja             |
| Paling cocok untuk | Kode yang eksplisit | Iterasi berbasis posisi | Mengambil kolom cepat |

**Kesalahan klasik:** `df.loc[0:2]` menghasilkan 3 baris, sedangkan `df.iloc[0:2]` menghasilkan 2 baris. Perbedaan ini sering menjadi sumber bug diam-diam.

## LATIHAN CEPAT

# Nama landmark apa yang dihasilkan?

1 `df.loc[1, 'Nama']`

2 `df.iloc[2, 0]`

3 `df['Nama'][3]`

*Diskusikan dengan teman sebelah selama 2 menit. Gunakan tabel pada slide sebelumnya sebagai acuan.*

## BAGIAN 04

# Penyaringan Data (Filtering)

*Mengambil baris yang memenuhi syarat dengan Boolean masking.*

- 1. Struktur Data pandas
- 2. Eksplorasi DataFrame
- 3. Seleksi & Pengambilan Data
- **4. Penyaringan Data (Filtering)**
- 5. Modifikasi DataFrame
- 6. Agregasi & Pengelompokan
- 7. Menggabungkan DataFrame
- 8. Visualisasi Data

# Boolean Masking: Menyaring dengan Kondisi

*Kita sudah bisa memilih berdasarkan posisi dan label. Bagaimana memilih berdasarkan syarat?*

## Cara kerjanya dua langkah

- 1 Kondisi menghasilkan Series berisi True / False sepanjang jumlah baris.
- 2 Series Boolean itu dipakai sebagai penyaring: baris bernilai True diambil, yang False dibuang.

```
df['Tinggi'] > 20
```

```
0    False
1     True
2     True
3    False
```

## Dua penulisan yang setara

```
df[df['Tinggi'] > 20]
```

*setara dengan*

```
df.loc[df['Tinggi'] > 20]
```

Versi .loc lebih dianjurkan karena dapat sekaligus memilih kolom dan menghindari peringatan SettingWithCopyWarning saat data diubah.

# Menggabungkan Beberapa Kondisi

Gunakan operator bitwise, bukan kata kunci `and` / `or` milik Python.

```
df[(df['Tinggi'] > 20) & (df['Jenis'] == 'Menara')]
```

| Simbol | Penulisan | Arti                                       |
|--------|-----------|--------------------------------------------|
| ~      | ~p        | Negasi dari p (kebalikannya)               |
| &      | p & q     | p DAN q — keduanya harus benar             |
|        | p   q     | p ATAU q — cukup salah satu benar          |
| ^      | p ^ q     | p XOR q — benar bila tepat satu yang benar |

## Wajib diingat

- Setiap kondisi harus dibungkus tanda kurung — prioritas & lebih tinggi daripada >.
- `and` / `or` akan memicu `ValueError` pada Series.
- Alternatif rapi: `df.query('Tinggi > 20 and Jenis == "Menara")`.

Perlu banyak nilai sekaligus? Gunakan `df[df['Jenis'].isin(['Menara', 'Plaza'])]` alih-alih menyambung banyak operator `|`.



## BAGIAN 05

# Modifikasi DataFrame

*Menambah, menghapus, mengurutkan, dan membersihkan data.*

- 1. Struktur Data pandas
- 2. Eksplorasi DataFrame
- 3. Seleksi & Pengambilan Data
- 4. Penyaringan Data (Filtering)
- **5. Modifikasi DataFrame**
- 6. Agregasi & Pengelompokan
- 7. Menggabungkan DataFrame
- 8. Visualisasi Data

# Menambah dan Mengubah Kolom

*Cukup rujuk nama kolom baru dengan [ ] lalu berikan nilainya.*

## Dari daftar nilai

```
df['Usia'] = [65, 50, 57, 40, 33]
```

Panjang daftar harus sama persis dengan jumlah baris, jika tidak akan muncul ValueError.

## Dari kolom lain

```
df['Tinggi_m'] = df['Tinggi'] * 0.3
```

Operasi berjalan tervektorisasi pada seluruh kolom sekaligus — tidak perlu perulangan.

## Sebagian besar operasi pandas TIDAK mengubah data aslinya

```
df.drop(columns=['Usia'])  
print(df)    # kolom masih ada!
```

Perintah drop mengembalikan salinan baru. Agar perubahan tersimpan, gunakan salah satu:

```
df = df.drop(columns=['Usia'])
```

# Mengurutkan Data dengan sort\_values()

*Mengurutkan memudahkan kita melihat nilai ekstrem dan pola.*

## Satu kolom

```
df.sort_values(by='Tinggi',  
               ascending=True)
```

## Beberapa kolom

```
df.sort_values(by=['Tinggi', 'Jenis'],  
               ascending=[True, False])
```

- Nilai bawaan ascending adalah True (menaik). Untuk menurun, tulis ascending=False.
- Pada beberapa kolom, urutan nama kolom menentukan prioritas pengurutan; panjang daftar ascending harus sama.
- Nilai NaN secara bawaan ditempatkan di akhir — atur dengan na\_position='first' bila perlu.
- Ingin mengurutkan berdasarkan label baris? Gunakan sort\_index().

# Menangani Nilai yang Hilang (Missing Values)

*Data dunia nyata hampir selalu tidak lengkap. Deteksi dulu, baru putuskan tindakannya.*

## 1 Deteksi

```
df.isnull()  
df.isnull().sum()
```

Menampilkan peta True/False, lalu menghitung berapa nilai hilang per kolom.

## 2 Hapus

```
df.dropna()  
df.dropna(subset=['Tahun'])
```

Aman bila jumlah baris hilang sedikit. Berisiko membuang informasi bila banyak.

## 3 Isi (imputasi)

```
df.fillna(0)  
df['Tahun'].fillna(  
    df['Tahun'].median())
```

Mengisi dengan nilai tertentu, median, atau modus. Pertahankan ukuran data.

*Tidak ada aturan baku. Selalu tanyakan: mengapa data ini hilang? Nilai hilang yang tidak acak dapat membuat model menjadi bias.*

## BAGIAN 06

# Agregasi & Pengelompokan

*Dari ribuan baris menjadi beberapa angka yang bermakna.*

- 1. Struktur Data pandas
- 2. Eksplorasi DataFrame
- 3. Seleksi & Pengambilan Data
- 4. Penyaringan Data (Filtering)
- 5. Modifikasi DataFrame
- **6. Agregasi & Pengelompokan**
- 7. Menggabungkan DataFrame
- 8. Visualisasi Data

# Fungsi Agregasi di pandas

*Agregasi meringkas banyak nilai menjadi satu nilai ringkas.*

## Dasar

```
sum()      ·    mean()      ·  
median()   ·    min()      ·  
max()      ·    count()     ·  
nunique()
```

## Statistik

```
std()      ·    var()      ·    sem()  
·    skew()    ·    prod()    ·  
quantile()
```

## Logika & Indeks

```
any()      ·    all()      ·  
first()    ·    last()     ·  
idxmin()   ·    idxmax()
```

### Contoh pada dataset kita

```
df['Tinggi'].mean()
```

→ 22.8

```
df['Tinggi'].std()
```

→ 31.2

```
df['Tinggi'].idxmax()
```

→ 1

# groupby(): Split — Apply — Combine

*Pola berpikir yang sama dipakai di SQL, Excel PivotTable, dan Spark.*

## 1 Split

Membagi baris ke dalam kelompok berdasarkan nilai suatu kolom, misalnya per Jenis landmark.



## 2 Apply

Menerapkan fungsi agregasi pada setiap kelompok, misalnya menghitung rata-rata tinggi.



## 3 Combine

Menyatukan hasil tiap kelompok menjadi satu tabel ringkasan yang jauh lebih kecil.

```
df.groupby('Jenis')['Tinggi'].mean()

# atau bentuk eksplisit:
df.groupby('Jenis')[['Tinggi']].agg('mean')
```

### Isi .agg() boleh apa saja?

Fungsi bawaan pandas ('mean'), fungsi NumPy (np.mean), fungsi Python (max), atau fungsi buatan sendiri — asalkan mengubah banyak nilai menjadi satu nilai. Beberapa sekaligus: .agg(['mean', 'max']).

# Mengelompokkan Berdasarkan Beberapa Kolom

*Bila hasilnya bertingkat dan sulit dibaca, ubah menjadi tabel pivot.*

## groupby dengan dua kolom

```
df.groupby(['Jenis', 'Kampus'])  
  [['Tinggi']]  
  .agg('max')
```

Hasilnya memakai MultiIndex: dua tingkat label baris. Informatif, tetapi kurang nyaman dibaca dan sulit dipakai langsung untuk grafik.

## pivot\_table: bentuk matriks

```
pd.pivot_table(  
    df,  
    index='Jenis',      # baris  
    columns='Kampus',   # kolom  
    values='Tinggi',    # nilai  
    aggfunc='max'
```

Satu kategori menjadi baris, kategori lain menjadi kolom. Bentuk inilah yang siap dipakai untuk heatmap atau grafik batang berkelompok.



## BAGIAN 07

# Menggabungkan DataFrame

*Menyatukan informasi dari dua tabel melalui kunci yang sama.*

- 1. Struktur Data pandas
- 2. Eksplorasi DataFrame
- 3. Seleksi & Pengambilan Data
- 4. Penyaringan Data (Filtering)
- 5. Modifikasi DataFrame
- 6. Agregasi & Pengelompokan
- **7. Menggabungkan DataFrame**
- 8. Visualisasi Data

# Mengapa Perlu Menggabungkan Tabel?

*Data nyata jarang tersimpan dalam satu tabel besar.*

## Tabel A — landmark

Nama · Jenis · Tinggi · Tahun

Berisi ciri fisik setiap bangunan.

## Tabel B — peristiwa

Tahun · Peristiwa

Berisi kejadian penting per tahun.

## Kunci penggabungan (join key)

Kolom Tahun pada tabel A dan kolom Tahun pada tabel B menjadi penghubung. Dari sinilah kita dapat bertanya: peristiwa apa yang terjadi pada tahun bangunan ini didirikan?

# Empat Jenis Join

*Perbedaannya hanya satu: baris mana yang dipertahankan ketika tidak ada pasangan.*



## Inner Join

Hanya baris yang kuncinya ada di KEDUA tabel. Paling aman, tetapi bisa membuang banyak data.



## Outer Join

Semua baris dari kedua tabel. Yang tidak berpasangan diisi NaN.



## Left Join

Semua baris tabel kiri, ditambah pasangan dari tabel kanan bila ada.



## Right Join

Semua baris tabel kanan, ditambah pasangan dari tabel kiri bila ada.

```
hasil = landmark.merge(peristiwa, how='inner', left_on='Tahun', right_on='Tahun')
```

# merge() atau join()?

*Dua perintah, satu tujuan — pilih sesuai bentuk kunci Anda.*

| Aspek              | df.merge()                                   | df.join()                              |
|--------------------|----------------------------------------------|----------------------------------------|
| Dasar penggabungan | Kolom mana pun (left_on / right_on / on)     | Index (kecuali dipaksa dengan on=)     |
| Fleksibilitas      | Paling fleksibel, jadi paling sering dipakai | Ringkas bila kunci sudah menjadi Index |
| Nilai bawaan how   | 'inner'                                      | 'left'                                 |
| Kapan dipilih      | Kunci berupa kolom biasa                     | Kunci sudah disetel sebagai Index      |

## Periksa setelah menggabungkan

Bandingkan df.shape sebelum dan sesudah. Jumlah baris yang membengkak menandakan kunci tidak unik (relasi banyak-ke-banyak); jumlah baris yang menyusut drastis menandakan kunci banyak yang tidak berpasangan.

## BAGIAN 08

# Visualisasi Data

*Mengubah angka menjadi pemahaman melalui gambar.*

- 1. Struktur Data pandas
- 2. Eksplorasi DataFrame
- 3. Seleksi & Pengambilan Data
- 4. Penyaringan Data (Filtering)
- 5. Modifikasi DataFrame
- 6. Agregasi & Pengelompokan
- 7. Menggabungkan DataFrame
- **8. Visualisasi Data**

# Mengapa Kita Memvisualkan Data?

*Dua tujuan yang berbeda, dan menuntut gaya grafik yang berbeda pula.*



## Insight — untuk diri sendiri

Menemukan pola, pencilan, dan hubungan antarvariabel. Grafik dibuat cepat, banyak, dan boleh berantakan. Inilah fokus mata kuliah ini.



## Komunikasi — untuk orang lain

Menyampaikan temuan kepada pembaca. Grafik dibuat sedikit, rapi, berlabel jelas, dan memiliki pesan tunggal.

*Kuartet Anscombe: empat dataset dengan rata-rata, varians, dan korelasi yang identik, tetapi bentuk sebarannya sama sekali berbeda. Statistik ringkas saja tidak pernah cukup — selalu gambar datanya.*

# Pustaka Visualisasi yang Perlu Anda Kenal

*Masing-masing punya tempatnya sendiri.*

## Matplotlib

Standar emas sejak 2005. Grafik statis, sangat dapat dikendalikan, dan masih dipakai untuk hampir semua gambar di jurnal ilmiah.

## Plotly

Grafik interaktif lintas bahasa. Bisa di-zoom, di-hover, dan disaring — cepat untuk eksplorasi. Fokus utama kuliah ini.

## Weights & Biases

Layanan untuk melacak proses pelatihan model dan menyimpan artefak eksperimen secara otomatis.

**Mengapa Plotly lebih dulu?** Karena interaksi mempercepat penemuan wawasan. Matplotlib tetap wajib dikuasai untuk laporan ilmiah.

# Memilih Jenis Grafik yang Tepat

*Pertanyaannya menentukan grafiknya, bukan sebaliknya.*

## Line Plot

Perubahan sepanjang waktu

## Bar Plot

Perbandingan antarkategori

## Histogram

Sebaran satu variabel numerik

## Scatter Plot

Hubungan dua variabel numerik

## Box Plot

Sebaran + pencilan per kelompok

## Heatmap

Pola pada matriks atau korelasi

*Aturan praktis: mulai dari histogram tiap kolom numerik dan bar plot tiap kolom kategorikal sebelum membuat grafik yang lebih rumit.*



# Tiga Cara Membuat Grafik dengan Plotly

*Dari yang paling sederhana sampai yang paling leluasa.*

1

## Fungsi plot bawaan pandas

TERMUDAH

Cukup panggil `.plot()` langsung pada DataFrame. Tempat terbaik untuk memulai.

2

## Plotly Express

MUDAH + EKSPRESIF

Mirip fungsi pandas, tetapi jauh lebih banyak pilihan grafik dan pengaturan.

3

## Graph Objects

LANJUTAN

Menyusun grafik dari komponennya, seperti Matplotlib. Perlu memahami konsep dasar Plotly.

*Ketiganya menghasilkan objek grafik Plotly yang sama, sehingga dapat disesuaikan lebih lanjut dengan cara yang sama.*

# Contoh Kode: pandas .plot() dan Plotly Express

*Perhatikan pola yang sama: data, jenis grafik, lalu pemetaan kolom ke elemen visual.*

## Lewat pandas

```
pd.set_option('plotting.backend',
              'plotly')

mpg.plot(
    kind='scatter',
    x='weight', y='mpg',
    color='origin',
    title='MPG vs Berat',
    width=800, height=600
)
```

Cukup satu baris pengaturan di awal notebook, lalu semua grafik pandas menjadi interaktif.

## Lewat Plotly Express

```
import plotly.express as px

px.scatter(
    mpg,
    x='weight', y='mpg',
    color='origin',
    size='cylinders',
    hover_data=mpg.columns,
    title='MPG vs Berat'
)
```

Yang kita tentukan selalu sama: jenis grafik, dan kolom mana dipetakan ke sumbu x, y, warna, ukuran, atau bentuk.

## LATIHAN CEPAT

# Apa yang terjadi pada penanda Menara?

```
px.scatter(df, x='Tahun', y='Tinggi', size='Tinggi',  
           hover_name='Nama')
```

Diketahui Menara memiliki Tinggi = 74 dan Tahun = 1975, sedangkan landmark lain jauh lebih rendah.

Bagaimana posisinya pada sumbu x dan y?

Bagaimana ukuran penandanya dibanding yang lain?

Apa yang muncul saat kursor diarahkan ke penanda itu?

# Ringkasan Pertemuan

*Delapan hal yang perlu Anda bawa pulang hari ini.*

1

Series, DataFrame, dan Index adalah tiga blok penyusun pandas.

2

Kenali data lebih dulu: `head()`, `info()`, `describe()`, `sample()`.

3

`loc` memakai label dan inklusif; `iloc` memakai posisi dan eksklusif.

4

Penyaringan memakai Boolean masking dengan operator `&`, `|`, dan `~`.

5

Sebagian besar operasi tidak in-place — tugaskan kembali hasilnya.

6

Nilai hilang perlu dideteksi dulu sebelum dihapus atau diisi.

7

`groupby` mengikuti pola split–apply–combine; `pivot_table` merapikan hasilnya.

8

Visualisasi bukan pelengkap laporan, melainkan alat menemukan wawasan.

# Tugas dan Cara Belajar Lanjut

*Latihan mandiri agar materi hari ini benar-benar melekat.*

## Tugas praktikum

- Ambil satu dataset publik (mis. dari data.go.id atau Kaggle).
- Lakukan eksplorasi awal: bentuk data, tipe kolom, dan nilai hilang.
- Buat minimal tiga penyaringan dengan kondisi majemuk.
- Buat satu ringkasan groupby dan satu pivot\_table.
- Sajikan tiga grafik Plotly beserta satu paragraf temuan untuk setiap grafik.

## Cara belajar mandiri

- Telusuri dokumentasi resmi pandas dan galeri Plotly Express — keduanya penuh contoh siap jalan.
- Saat menemui kebuntuan, tanyakan pada asisten AI; model bahasa sangat baik untuk pandas dan Plotly.
- Namun jangan menyalin mentah-mentah: jalankan kodenya, ubah satu argumen, lalu amati apa yang berubah.
- Kemampuan ini hanya tumbuh lewat pengulangan pada data yang benar-benar Anda pedulikan.

TERIMA KASIH

# Perkakas Data

Pertemuan berikutnya: eksplorasi data lanjutan dan pra-pemrosesan untuk pemodelan.

*Materi diadaptasi dari kuliah “Data Tools” oleh Joseph E. Gonzalez dan Narges Norouzi, disusun ulang dalam bahasa Indonesia. Topik ini tidak dibahas pada buku teks utama.*